

复杂金融文档还原挑战·方法描述

超长/超大金融文档截图 → 高保真 Markdown

参赛队伍名称

2026-08

1. 任务与核心判断

赛题要求把超长/超大金融文档截图还原为高保真 Markdown，评分由文本编辑距离、表格 TEDS、阅读流编辑距离三项等权构成。唯一可用的视觉语言模型是主办方提供的 FinixDoc-VL API，不可微调、不可替换。

1.1 底座特性实测

方案设计的前提是先摸清这个黑盒的行为边界。我们在正式开发前用标定实验测得四条硬约束：

特性	实测结论	对方案的约束
接口形态	请求体仅 <code>userId / apiKey / fileName</code> + 图片；传 <code>prompt/query</code> 返回“无法查询智能体配置”， <code>text/instruction</code> 超时	无法用提示词控制输出格式、行结构或空格处理，全部结构控制必须在客户端完成
单次输出上限	约 10–12k 字符，与输入图像尺寸无关	每张表所需调用次数 $\approx \lceil L_{GT}/10^4 \rceil$ ，由内容容量而非像素量决定
内部分辨率上限	约 1500 px；更大的图会被内部降采样	超过该尺寸的输入，小字会被糊掉，必须在客户端切到 1500 px 以内
并发能力	16 路并发无限流，延迟仅从 10s 升到 13s	时间预算不是瓶颈，可放心用细粒度切分切换精度

其中第三条最具决定性。它意味着任何保留全宽的切分方式都是无效的——把一张两万像素宽的表按行带切开，宽度方向仍然超限，密集数字必然糊掉。

1.2 方法论

上述约束共同指向一个结论：模型本身没有差异化空间，全部空间在切什么、切多大、结果如何校验与修复。

由此确立本方案的核心原则：**把结构决策权从模型手中收回到代码中**。几何负责定位——切在哪里、有几行几列、哪些位置是空的；模型只负责回答“这个位置上是什么内容”。

这个划分的意义在于错误的**可能性边界**。视觉语言模型采用自回归解码，在表格上需要依靠自身已生成的输出来追踪当前所处的行列位置；当内容高度重复时（精算表中连续出现的相同数值），这种自追踪会失效，表现为跳行、复读、整块展平。常规应对是增加容错（行数容差、多次采样投票），但容错只能降低错误概率。若行列位置由代码给定、模型每次只回答单个格子的内容，则“数错行数”这一类错误的**发生条件本身不存在**。

2. 数据特征

两类图的物理形态与失败模式完全不同，必须分两条流水线处理。类别由官方发布目录直接给定，不做自动分类——数据集本就按两类分目录发布，用长宽比推断只会平白引入一类错误。

维度	LONG (面条图)	TABLE (大表图)
数量	训练 100 / B 榜 50	训练 100 / B 榜 50
宽度	锁定 1500–1508 px	3306–22928 px
高度	3 万–19 万 px	3306–23390 px
像素量	46–287 M	15–387 M
长宽比	20–128	约 1.4
GT 格式	纯 Markdown (#-##### + 段落), 中位 1.6 万字符	HTML <table><tr><td>, 最高 63 万字符

TABLE 的 GT 字符量分布极度右偏: 最小 5k、中位 74k、均值 134k、90 分位 425k、最大 639k。按每次调用 10k 字符的上限折算, 中位表约需 8 次调用, 90 分位表约 43 次, 最大表约 64 次。50 张测试表合计约 694 次调用, 而 50 张 LONG 仅约 89 次。**TABLE 是计算量与难度的双重主体。**

3. 系统总览

输入按 long/ 与 table/ 两个目录分流到两条流水线, 各自产出的 Markdown 合并后按 file_name 排序写出 submission.csv。

LONG 空白带切条 --> API 并发识别 --> 接缝拼接 --> 文本层级校正 --> 几何层级定级
slicer_long api_client stitch_long heading_norm geom_heading

TABLE 几何裁剪 --> 网格切分 --> API 并发识别 --> 一致性判定 --> 确定性重读 --> 二维重组
crop slicer_table api_client grid_ocr cell_ocr stitch_single

图级并行由进程池承担 (绕开 GIL, 几何计算是 CPU 密集型), 图内的 API 调用由线程池并发。单图异常不影响整批。

4. TABLE 流水线

TABLE 路是本方案的主体。整体分三个阶段: Stage I 用纯几何手段把整图分解为若干单子表; Stage II 在每个子表内按网格线切分并识别, 同时对每个切块做一致性校验与修复; Stage III 把校验通过的切块按全局坐标重组为完整表格。

4.1 Stage I: 几何裁剪与子表切分

动机来自评测口径。官方 TEDS 按出现顺序对 GT 与预测中的 <table> 逐个配对。若一张图上有 N 个并列表而系统将其读成一个大表, 则只有 GT 的第一个表能被配上, 其余全部计零分。**输出的表数量必须精确等于 GT, 差一个就会造成全盘错位。**

这一阶段完全不调用模型, 输出是一个有序的块序列 $[(k_i, b_i)]$, 其中 $k_i \in \{\text{text}, \text{table}\}$ 为块类型, b_i 为包围盒。分三层:

第一层, 剥离表外文字。页眉、页脚、水印、页码等元素若混入表格区域会污染网格估计。判据是三个条件的合取: 块高度低于阈值 (矮)、块宽度显著小于主体 (窄)、且非最大连通分量。三者缺一不可——“窄”这一条尤其关键, 它保护了三角形稀疏表底部那些又宽又矮的有效数据段不被误剥。

第二层, 切分并排子表。早期版本使用“段数少于 4”之类的经验阈值, 既不可解释也不稳定。最终采用两条几何判据的合取:

判据一, 缝高倍数。定义整行墨量低于宽度 0.3% 的像素行为白线, 连续白线构成白带。设白带高度的中位数为行缝基准 h_0 , 则真子表缝需满足

$$h_{\text{seam}} \geq k \cdot h_0, \quad k = 2$$

k 的取值有明确的数据依据: 实测真子表缝为行缝的 2.5–10 倍, 单表内空行造成的缝约为 1 倍, 两个分布之间在 [1.1, 2.5] 区间内几乎没有样本, $k = 2$ 落在这个空隙中央。此处不使用 Otsu 等自适应阈值——单表内所有行缝增高时 Otsu 会退化, 把每一条行缝都当作切点。

判据二, 框线连续性。仅靠墨量无法区分“表内空行”与“真子表边界”: 稀疏表底部的空行虽然墨量为零, 但表格的边框与列线仍连续穿过。因此要求缝内贯穿的竖线条数不超过 2。实测真子表缝处框线断开 (贯穿数为 0), 而表内空行处仍有多条竖线贯穿。

第三层，标题段归类。子表上方的标题文字被空行隔开，形成前导矮段。这类段必须单独标记为 `text` 类型——任何送入表格识别流程的块都会被包装成一个 `<table>`，标题段会凭空产生一个空表，使表数量加一，触发前述的全盘错位。标题段与表外文字共用同一套纯文本识别路径。

4.2 Stage II：网格切分

在单个子表内，切分必须严格落在检测到的网格线上，使每个切块（tile）包含**整数行、整数列**。这条约束直接消除两类密表病态：切点落在行内会导致边缘半行的取舍在相邻块之间不一致（行数相差一行）；又宽又矮的薄条会使模型无法辨识行边界，把整块内容压进单个 `<tr>`（曾观察到单块输出 1 行 × 819 列的极端展平）。

切块尺寸上限为 25 行 × 15 列，由单变量隔离实验确定。

空白块预判。墨量极低的切块不送模型，直接按已知网格填充空单元格。这既节省调用（100 张图上省下约 1125 次），也根除了一类幻觉——由于无法用提示词告知模型“此处为空，请勿编造”，唯一可靠的做法是不把空白送进去。

小字上采样。定义切块的等效格边长

$$e = \sqrt{\frac{A_{\text{eff}}}{n_r \cdot n_c}}$$

其中 A_{eff} 为去除页边距后的面积， n_r, n_c 为行列数。当 $e < 40$ px 时判为密集小字，放大倍数取

$$s = \text{clamp}\left(\frac{75}{e}, 1, 3\right)$$

即把格边长调整到目标值 75 px。使用二维格面积而非一维列间距，是为了正确识别“列稀行密”型表格。

这里有一个反直觉但可复现的实测结论：**密集小字块在原始分辨率下，模型不仅识别更差，速度也更慢**——单块耗时 181 秒且输出行幻觉与列漂移，上采样 2.58 倍后仅需 45 秒且识别正确。合理解释是模型在超出其有效分辨率的输入上会反复挣扎、走入慢速解码路径。因此上采样并非“以时间换精度”的取舍，而是精度与延迟的同时改善。

4.3 一致性判定与确定性重读

这是 TABLE 路的核心机制，也是最终版本相对前代的主要增益来源。设计遵循三段式：**几何期望** → **一致性判定** → **确定性重读**。

几何期望。对每个切块，由墨迹几何独立计算出一个期望结构，不依赖模型输出。设 $\text{ink}(i, j)$ 与 $\text{gray}(i, j)$ 分别为严阈值与松阈值下格 (i, j) 的墨迹指示（严阈值取灰度 128，捕获正文浓墨；松阈值取 180，兼容淡灰框线与浅色标题），定义格占位

$$\text{cell}(i, j) = \text{ink}(i, j) \vee \text{gray}(i, j)$$

期望结构 E 即由全部 $\text{cell}(i, j)$ 构成的二值占位图：哪些行含有内容、每行的哪些格有墨。

一致性判定。模型返回的解析结果需满足**占位恒等**：非空行数等于 $|E|$ ，且逐格满足

$$\forall(i, j) : \quad \text{非空}(i, j) \iff \text{cell}(i, j)$$

判定为零容差——不允许众数表决，不允许 ± 1 行的容差。

这一判据是逐步收紧演化出来的，中间两个版本的失效方式说明了为什么必须做到逐格：

- 版本一，列数上界。仅要求解析出的列数不超过几何列数 n_c 。这在 $[\text{有效宽}, n_c]$ 之间留下了一个区间，模型的重复输出（同一值被连续吐出多次）可以藏身其中而不被察觉。
- 版本二，宽度恒等。要求每行的有效宽度等于几何有效宽度。这仍然只是占位图的一维投影，无法发现“前段或中段占位缺失、右侧有值”这类形态——行的总宽度不变，但内容整体错位。

占位恒等是这一系列判据的完全形式，宽度恒等只是它的推论。理论上仍不可见的残余情形是“占位完全相同、仅数值被读错”，这需要内容层面的证据（数值平滑性验证）才能捕获，属于未来工作。

确定性重读。判定不通过的切块，触发唯一一个修复动作：整块回落本地 PP-OCRv6s 逐格识别。

过程 `REPAIR(tile, 骨架行边界 rb, 列边界 cb, 期望 E)`：

```

对每个满足  $E(i,j)$  为真的格  $(i,j)$ :
    crop ← tile 在 [cb[j], rb[i], cb[j+1], rb[i+1]] 处的裁剪
    text[i,j] ← 本地识别(放大 3 倍的 crop)      # 结果按内容哈希缓存
对  $E(i,j)$  为假的格: 置空
返回 由 text 装配的规整网格

```

行列位置完全由几何骨架给定，本地模型只回答“这个格子里是什么”。位置控制权收归代码后，计数类错误（行数爆炸、漏行、跳读）在结构上不可能发生。值得注意的是，单调重复区（同一数值连续出现数十次）恰恰是 API 最容易失稳、而逐格识别最容易处理的输入类型。本地模型输出确定，重放结果天然稳定。

这一机制的架构意义在于：它用一个**通用修复动作**替换了此前针对各类病态分别编写的七个专用修复模块，代码量减少约 370 行，而覆盖面反而更完整。判据越严格，抢救逻辑就越少——这是零容差判定相对容错判定的根本优势。

4.4 Stage III: 二维重组

重组把各切块的识别结果按全局行列坐标装配为完整表格。主要处理四类结构问题：

全局列宽一致。各横带独立重组时，若真实块使用模型返回的列宽、空白块使用几何检测的列宽，二者不一致会导致整表参差不齐，TEDS 崩溃。解决办法是为每个切块列 c 确定一个规范列宽 W_c （该列各横带非退化块列宽的众数），所有横带的第 c 块统一对齐到 W_c ，使每行的总列数恒为 $\sum_c W_c$ 。

跨列表头的 colspan 合成。表头行中的文字往往横跨多个数据列。按内容簇分段：每个文字格向右吞并连续空档直到下一个文字格，行首空档并入首格，生成对应的 colspan 属性。这里设有**数据行守卫**——训练集中带尾部空单元格的行首全部是以数字为主的数据行，GT 保留空 `<td>` 而非合成 colspan；因此当非空格中数字格占多数时不做整形。

表头结构重建。以“列号行”为锚点：在前三行中寻找连续整数游程长度不小于 5 的行，其上一行按内容形态分三种情况处理——单个短簇折入下方角格形成斜线表头，全为短簇则识别为双行表头（左侧标签上下合并为 rowspan，右侧标签簇发 colspan），含长簇（超过 10 字）则整行提升为表格标题。列号行以上的所有行一律标题化。

编号序列补齐。表格中的行号列与列号行常因识别丢失而出现空洞。按局部连续段做线性补齐（横向限前 3 行、纵向限前 4 列，仅补 0–9 范围）。必须按局部段而非全局仿射拟合——并排双面板的两段列号行截距不同，全局拟合会同时算错两段。

5. LONG 流水线

LONG 路的结构问题远轻于 TABLE：面条图宽度锁定 1500 px，恰好落在模型的有效分辨率内，横向不需要切分。核心挑战是切点选择、接缝处理与标题层级。

5.1 空白带切分

不做等高裸切，而是求每行墨量的水平投影，在**行间空白带**下刀，使切点落在文字行之间。这直接消除了接缝处的半行残字——后者是阅读流与文本编辑距离的主要失分源。目标条高 5000 px，依据是实测延迟随高度线性增长（1500 px 约 8 秒，5000 px 约 13 秒，10000 px 约 27 秒），5000 是延迟与条数的平衡点。

表格避让。条带内若检测到表格，切点应绕开，避免把一张表切成两半。表格检测按“线”而非按像素列计数（一条框线通常占 2 个相邻像素列，按列计数会高估一倍），且逐带独立核验，要求贯穿竖线不少于 3 条（左边框 + 列线 + 右边框）——阈值取 2 会把双栏排版的分栏线误判为表格。

避让规则附带一条重要约束：**带高超过目标条高的表不避让**。这条约束是逐项隔离实验逼出来的。早期版本对所有检测到的表都避让，结果对那些本来就放不下的大表，避让买不到完整性，反而顶高切点、造出异常短的条带并引发连锁位移。合并测试时这一改动看似有效（TEDS 提升 0.64），单独隔离后才发现是净负——阅读流下降 0.37、文本下降 0.06，且 TEDS 只覆盖含表的 48 篇而另两项覆盖全部 100 篇。加上高度约束后转为正收益。

5.2 接缝拼接

拼接需处理两类情形：

普通接缝。相邻条带以不重叠的坐标裁出，正文接缝只需续行、不应删除内容。早期版本使用多行平均相似度做模糊去重，但该度量会被空行与保险条款的套话抬高，曾整段删除有效正文。最终改为不做模糊去重，表内的重叠由专门路径处理。

跨条表格。长表格中部没有空白带，切点必然落在表内，导致上一条被收口为 `</table>`、下一条重新开启 `<table>`。此处需要跨条重连。另有一类情形是切点穿过某个长单元格，使该格的前半部分被读到表外形成孤儿文本——识别条

件为：上条已闭表、其后的孤儿文本很短且以连接标点结尾、两侧表格列数相同，满足时将孤儿前缀塞回下条首行的末单元格。

5.3 标题层级：文本相对定级与几何绝对定级

模型按条带识别，# 的层级是各条带内的局部判断，拼成整篇后同一编号序列常在条带边界被重置。层级处理分两级，先文本后几何。

文本相对定级 (heading_norm) 采用栈模型：维护一个大纲栈，按编号格式的连续性判断当前标题是同级、下钻还是回到上级。关键修正是——当遇到一个此前出现过的编号序列时，采用该序列的历史层级，而非相邻标题层级加減一。后者会在“深层游走后回到浅层”的情形下持续漂移。

此外还有三条针对性处理：约 56% 的文档中，封面大标题被模型拆成若干无 # 的普通段落，需合并并提升为一级标题；目录条目常被输出为 * 1.1 ... 形式的列表项，需在目录区转为标题；表内满宽横幅行的 colspan 需归一化（训练集 GT 中该写法有 102 例，写作裸 <td> 的仅 5 例）。

几何绝对定级 (geom_heading) 是 LONG 路的最后一步，也是全流程中唯一回到原图做全局裁决的一步。标题层级本质上是排版信息，前述所有步骤都只能从文本形态间接推断。这一步回到原图逐行测量：

- 字号：取行内 core 高度（墨量不低于中位数 0.6 倍的像素行数），相对本文正文中位数归一，每个文档独立计算；
- 淡横线：检测标题上下的浅色分隔线。

裁决规则是：淡横线上方判为章级一级标题；无编号的中段标题按字号判定，不低于正文中位数 1.25 倍者保留为一级标题，否则判为模型误报并删除该标题标记。逐行识别使用同一个本地小模型，结果按图片名缓存。

这一步必须放在拼接与文本定级之后——只有整篇拼合完成，字号才具有全局可比性。

5.4 关于标题层级的一个重要发现

我们曾长期认为标题层级是 LONG 路的主要失分点，并投入大量精力。一次隔离提交推翻了这一假设：仅修改标题层级，共 13 篇文档发生变化、627 个标题的层级数被改变（占 B 榜 LONG 全部 2763 个标题的 22.7%），非标题正文一字未改，线上分数完全没有变化。且这 13 篇中有 6 篇属于混合位移，父子层级关系确实被重排，并非整体平移。

该零结果的分辨率是充分的：若阅读流按逻辑块计算且计入 #，627 个标题约占全部块的 4.2%，应当带来约 0.63 分的变动。结论是 # 标记本身不进入评分，层级关系亦然。

这一发现并未使标题处理链路失去意义——它改变了该链路的定位。上述处理中，合并被拆散的封面标题、把目录列表项转为标题行、删除模型误报的标题，这些操作都改变了文本内容本身，而文本内容是计分的。真正无效的只是层级数字的调整。此后我们把 LONG 的投入转向了实际的失分源：残余失配块中约 41.4% 是形近字识别错误，约 29.4% 是内容在 GT 中原样存在、仅断行位置不同。

6. 参数设置

参数	值	依据
MAX_CONCURRENCY	32	16 为标称上限，实测 32 吞吐更高；触发限流时自动退避且不写入缓存
POOL_PROCS	6	图级进程并行绕开 GIL；几何计算为 CPU 密集型
API_TIMEOUT	40 s	服务端过载时表现为“收下不回”，超过 40 秒继续等待无收益
API_RETRIES	3	配合快速放弃策略；失败项由缓存重跑补齐
DEGEN_RETRIES	1	复读退化是确定性的（同一输入删缓存连续调用 5 次全部复现），多次重试无收益
LONG target_h	5000 px	延迟随高度线性增长，此处为延迟与条数的平衡点

参数	值	依据
LONG MIN_RULES	3 条	表格至少含左边框、列线、右边框；取 2 会把双栏排版误判为表格
TABLE 切块上限	25 行 × 15 列	单变量隔离实验确定
上采样阈值 / 上限	$e < 40 \text{ px} / 3\times$	目标格边长 75 px；二维格面积可识别“列稀行密”型表格
子表缝高倍数 k	2	真缝为行缝的 2.5–10 倍、空行缝约 1 倍， $k = 2$ 落在两分布的空隙中央
缝内贯穿竖线上限	2 条	真子表缝处框线断开，表内空行处框线仍连续贯穿
BIN_INK / BIN_FAINT / BIN_LINE	128 / 180 / 170	正文为浓墨；框线与浅色标题多在灰度 130–180，故分严松两档

全部阈值均为几何或统计维度上的通用判据，不含针对特定图像的分支或白名单。

7. 迭代历程

TABLE 路经历了三次架构级迭代，LONG 路的演进相对平缓。以下分别叙述。线上分数按分半提交口径给出 (TABLE 与 LONG 分别提交、另一半置空，因而可分别归因)。

7.1 TABLE 第一代：整表切分 —— 约 80 分

第一代把每张图当作单个表格处理：检测网格线，按网格切分为切块，逐块识别后按坐标拼回。这一代的起点是一个决定性的实验结论。最初的行带切分（只切高度、保留全宽）在中位表上切了 16 个带，TEDS 仅 0.49，内容只读回 62k/74k 字符——原因正是宽度超过模型内部分辨率上限而被降采样。改为原生分辨率的二维方块切分后，内容召回率的对比如下：

切块尺寸	中位表切块数	内容召回率
1100 px	35	0.979
1500 px	20	0.998
2000 px	12	0.920
2400 px	6	0.808

1500 px 锁定为切块上限。至此问题从“内容读不回来”转化为“读回来的碎块如何拼成正确结构”。在这一架构上我们做了十余轮改进：多子表分段（利用模型自身返回的多个 <table>）、列数封顶、超大切块强制细分、截断表恢复（补齐未闭合标签而非整块丢弃）、展平幻觉过滤、空白判据阈值调整、全局列宽一致、尾部全空列裁剪等。本地全量交叉验证的综合分从 71.1 提升至 83.2，线上 A 榜达到 80.44（其中密集小字上采样是单项最大增益）。**瓶颈与放弃的原因。**到 83 分附近，改进变得越来越困难且互相冲突——修好一类表往往打坏另一类。根本原因是这一代**没有可靠的行列估计**：所有结构决策都依赖模型返回的行列数，而模型在密表上的行列分割本身就不稳定（同一横带的各列块可能读出 11、12、0 等互不相同的行数）。在一个不可靠的基准上做修补，只能不断增加特例分支，越改越乱。

7.2 TABLE 第二代：全面几何化 —— 约 92 分 (TABLE 52 + LONG 40)

第二代是架构重写，核心是把结构决策从模型转移到几何：

- 剥离全部表外文字**：页眉、页脚、水印、页码，以及子表标题，一律先切出来单独走纯文本识别；
- 子表切分**：用缝高与框线连续性两条判据把整图切成若干单一子表，每次只处理一个；
- 行列估计**：在单个子表内独立完成行列网格的几何估计，作为后续所有装配的坐标基准。

第 2 项的效果最为显著：整体本地分从 84.07 提升至 88.37，其中被切分的多子表从 69.66 提升至 83.94（提升 14.28），而未被切分的单表输出逐字不变——这是一个零风险的改动，因为判据不满足时流程完全退化为原路径。

行列估计的精度在这一代达到可用水平：有框表的列估计 51/52 精确，无框表的行估计精确率从 77/113 提升至 106/113、列估计从 23/96 提升至 84/96 且零多估。

残留问题。即使行列估计完全正确，模型返回的内容仍可能与之不符，且失配形态多样：幻读（凭空产生重复内容）、展平（整块压成一行）、少行、少列。第二代针对每一种形态分别编写了修复逻辑——对半重读、截断补救、展平三路抢救、塌缩拆格等，共七个模块。这些模块彼此耦合、触发条件互相干扰，且每种都只覆盖已知形态，遇到新形态就失效。

7.3 TABLE 第三代：统一的一致性判定与确定性重读 —— TABLE 53.34

第三代的洞察是：上述七种失配形态是**同一个问题的不同表现**——模型返回的结构与几何期望不一致。既然如此，就不需要分别识别每种形态，只需要一个判据判断“是否一致”，和一个动作处理“不一致”。

于是有了 4.3 节的三段式：几何期望给出零容差的占位恒等判定，不通过则整块回落本地小模型逐格重读。七个专用修复模块全部删除，代码减少约 370 行。

线上 TABLE 半边达到 **53.34 / 55**，平均 TEDS 约 97%。在 B 榜集合上，223 个切块触发了本地重读，残余未解决的审计告警仅 4 条。

7.4 LONG 路的演进

LONG 路的基础流水线（空白带切分 + 接缝拼接）在早期就达到了较高水平，本地综合分约 92.5 且长期无回归。后续改动集中在两处：

标题处理。依次实现了栈模型相对定级、封面标题合并提升、目录列表项转标题、几何绝对定级。如 5.4 节所述，其中层级数字的调整经隔离实验证明不计分，实际增益来自那些改变文本内容的操作。

长文内表格。面条图中散布着表格，而表格在评分中权重较高。这方面的改动包括表格避让切点、跨条表格重连、孤儿单元格回收、满宽横幅行的 colspan 归一。这一批改动使线上 LONG 半边从 40.49 提升至 **41.99 / 45**，训练集上的表格 TEDS 从 95.27 提升至 96.69，方向一致。

7.5 一个方法论层面的教训：先验证评测工具

本地 TEDS 实现中曾存在一个严重缺陷：行对齐使用“行签名精确匹配”，而预测行因识别误差或列数差异**永远不可能**等于 GT 行签名，实际退化为按位置配对；一旦行数不等，缺行或多行之后的所有行全部错位，距离急剧增大。一张仅多两列、内容完全正确的表被评为 0.37，修复后为 0.911。

修复方案是改用带状行级动态规划，替代代价取单元格级编辑距离，与 APTED 口径对齐；GT 与自身比较仍精确等于 1.0。修复后重新分桶发现，此前占 17 张的“单元格错位”类问题大部分是指标假象。

这个缺陷使若干轮改进的评估结论失真，也让我们一度朝错误方向优化。在评测口径不完全透明的比赛中，先验证评测工具本身的正确性，其价值高于多写几条规则。

8. 否决方案记录

以下方案均经实测否决，记录在案以说明参数与结构选择的来由：

方案	结果	原因
用检测到的网格骨架 强制 约束模型返回的行数	均值 0.38 → 0.09，回退	当时的行列线检测精度不足（83 列对真值 70），硬性约束把原本正确的切块也搅乱。 检测结果可作软先验，不可作硬约束
裁剪三角表尾部空白列	净均值 0.47 → 0.45，回退	GT 的参差程度逐表而异：帮助了参差型表（0.25 → 0.36）却损害了保留空单元格的表（0.89 → 0.83），无通用规则
竖线缝拆分有框多子表	79.2 → 79.0，弃用	过度切分
横带行数取最大值以恢复截断行	79.9 → 79.8，回退	恢复出的是空行，内容已在截断时丢失

方案	结果	原因
五种子表检测（固定间隙 / 覆盖度 / 自适应 / 降采样问模型 / 列指纹）	全部不稳健	要么欠切要么过切； 过切会破坏当时已达 0.7–0.95 的 74 张单表 。根因是三角表的自然收窄与真子表边界在投影上不可区分
LONG 路几何分表	弃用	全训练集仅 1 篇存在真实损失，上限 +0.93 TEDS，却需改动全局表格装配（现有 45/48 篇正确）
全局标题层级偏移校正	两个候选端到端均更差	26/100 篇整篇差一级、上限 +11 分，但常数锚点与最浅归一两种方案都失败；无原理支撑则不强改
全角/半角标点归一	全部为负	NFKC 口径下不计分；若计分则占残余编辑量 56.8%，但所有规则（全转全角、全转半角、汉字紧邻、文档多数派）在字级与块级下均为负收益——GT 中汉字紧邻的标点本身有 17.6% 是半角
截断块重新识别 列校准投票采纳	停用 废除采纳，仅保留审计	全集耗时约 3 小时，违反时间预算 均匀重复内容会骗过投票，产生空列与幻读列

9. 推理资源与时间

不依赖 GPU。全流程为 CPU 计算、远端 API 调用，以及一个 5.27M 参数的本地 CPU 小模型。

项	配置
硬件	24 核 CPU / 62 GB 内存 / 无 GPU；建议不低于 8 核、16 GB
软件	Python 3.11 / Linux；依赖见 requirements.txt，无需编译扩展
磁盘	不低于 2 GB（API 结果与本地识别结果的落盘缓存）
并发	图级 6 进程 × 图内 32 线程

推理时间：B 榜实测两个半边各约 10 分钟（各 50 张，并发 32）。引入本地小模型承担残差重读与几何定级后两侧均略有增加——几何定级冷启动约 5 秒/张，残差重读仅在判定不通过的切块上触发。全量 100 张按 30 分钟预估，留足余量亦在 1 小时以内，远低于 3 小时上限。

耗时的主要变数在 API 侧负载而非本地计算，这也是 API_TIMEOUT 设为 40 秒快速放弃、依靠缓存重跑补齐失败项的原因。

未使用 ensemble。全流程只有 FinixDoc-VL 一个视觉语言模型，无多模型投票、无多次采样融合。本地小模型不参与投票，仅在一致性判定不通过时作为确定性执行体接管，二者是串行的职责划分而非并行的结果融合。全部增益来自几何前处理与结构后处理。

10. 主要创新点

- 其一，**把概率性错误转化为结构性不可能**。面对模型在密集表格上的跳行与复读，通行做法是增加容错与冗余投票，这只能降低错误率。本方案通过几何骨架固定行列坐标、让模型退化为无状态的单格问答，使计数类错误的发生条件不复存在。这是本方案与常规纠错思路的根本区别。
- 其二，**零容差判定优于容错判定**。任何容差都会留出让错误藏身的窗口，且每种容差只针对一种已知形态。占位恒等判定没有窗口——要么完全一致，要么整块重读。其直接后果是抢救逻辑的大幅简化：七个专用修复模块被一个通用动作取代。**判据越严格，系统越简单**，这与“复杂问题需要复杂方案”的直觉相反。
- 其三，**接口约束反向催生了更优的前处理**。由于无法用提示词告知模型“此处为空，请勿编造”，我们必须用墨量投影自行预判空白块并跳过，结果既节省了大量调用又根除了一类幻觉。这个方案比“能够使用提示词”时会采取的方案更为鲁棒。

其四，输入适配是双赢而非取舍。密集小字在原始分辨率下模型既慢又错，上采样后既快又准。这提示视觉语言模型存在一个“有效分辨率舒适区”，把输入调整到该区间内，精度与延迟可以同时改善，不必在二者间权衡。

其五，隔离实验作为基础设施。多项改动打包评估会因互相抵消而得出错误结论——LONG 路的表格避让单独测量时是净负，打包测量时看似有效。我们坚持每次提交只改动一个变量，使线上分数变化可归因。标题层级的零结果实验用一次提交排除了整个方向的投入，其价值不低于一项正向改进。

11. 未来可拓展方向

近期可实施：

1. **形近字识别精度**。残余失分中约 41.4% 是形近字错误，这类错误对 API 而言是系统性的。可利用本地小模型做分歧检测而非全量替换：仅在本地识别结果与 API 返回不一致的单元格上触发仲裁，成本可控且不影响整体吞吐。
2. **断行位置对齐**。约 29.4% 的失配内容在 GT 中原样存在，仅断行位置不同。这不是识别问题而是排版还原问题，可利用行间距的统计分布区分“同段续行”与“新起段落”。
3. **评分权重的实验标定**。提交一个剔除全部 `<table>` 的版本与原版对比，可直接测出表格项在总分中的实际权重与去向。目前本地指标可见的改进余量只有线上实际缺口的三分之一，约有 3.5 分的差距在任何本地指标上都不可见，需要用这类实验把它变得可测。

中期方向：

4. **切块锚定表头与标签列**。切分时为每个切块附加所属表格的首行与首列，使其成为自治的子表。这一改动可同时缓解四类问题：行折叠（标签列使每行非空）、列折叠（表头使每列非空）、列爆炸（按表头去重跨带重复列）、行列对齐（以标签与表头为键）。实测证据是带标签列的切块中，模型会保留空数据行而非将其折叠。代价是每个切块增加一行一列的开销。
5. **结构先验作为验证器**。精算表的数值沿行或列通常平滑或单调，离群值大概率是识别错误，可裁出重新识别。这正是 4.3 节所述“占位相同、仅数值错误”这一理论残余的补集，需严格限定只处理高置信的结构性错误，绝不臆造内容。
6. **版面分析扩展**。当前多栏处理依赖几何投影，对图文混排、跨页表格延续等复杂版式仍有提升空间，且可在 10M 参数预算内实现轻量版面模型。

方法论层面：本方案的核心模式是“几何定位 + 模型识别 + 零容差校验 + 确定性兜底”。这一模式不限于文档解析——任何“大模型在长序列上会丢失位置追踪”的任务，如长表格生成、长列表处理、多轮结构化抽取，都可以用同样的思路把位置控制权交还给代码，把模型的职责压缩到它真正稳定的局部判断上。

12. 合规声明

- **大模型使用**：全工程唯一的视觉语言模型接口是主办方提供的 FinixDoc-VL API，调用点集中于 `src/common/api_client.py`（两处 `requests.post`），无任何其它第三方模型 API、SDK 或网络地址。
- **提示词**：该 API 请求体仅含 `userId / apiKey / fileName` + 图片文件，不接受文本提示词参数，因此本方案不存在提示词配置文件。
- **本地小模型**：PP-0CRv6_rec_small.onnx，5.27M 参数（低于 10M 上限），CPU + onnxruntime 推理。同一引擎初始化时载入的检测与方向分类模型在调用时均置 `use_det=False / use_cls=False`，不参与推理；三者参数量合计亦仅 7.72M。模型文件随 rapidocr 安装包一并提供，运行期不联网下载。
- **无结果硬编码**：代码中不含针对特定测试图像的固定输出、白名单或标识符分支。
- **一键复现**：`bash run.sh` 单条命令产出完整的 `submission.csv`。